Міністерство освіти і науки України

Національний технічний університет України

«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Лабораторна робота №5

з дисципліни «Об'єктно-орієнтоване програмування»

на тему:

«Розробка багатовіконного інтерфейсу користувача для

графічного редактора об’єктів»

Виконав:

студентка групи ІМ-55

Горбань А.О

Номер в списку: **6**

Перевірив:

Рекечинський Д.О

Київ - 2026

Завдання:

1. Створити у середовищі розробки проект з ім’ям Lab6.
2. Написати вихідні тексти усіх програм-компонентів згідно варіанту завдання.
3. Скомпілювати вихідні тексти і отримати виконувані файли програм.
4. Перевірити роботу програм. Налагодити взаємодію програм.
5. Проаналізувати та прокоментувати результати та вихідні тексти програм.
6. Оформити звіт

Розрахунок варіанту:

Номер в списку: 6

6(mod 4) = 2 (Варіант 2)

2	1. Користувач вводить значення параметрів **n**, **Min**, **Max** у діалоговому вікні.	1. Створює вектор **n** дробових **(double)** чисел у діапазоні **Min – Max**	1. Зчитує дані з буферу обміну 2. Виконує

Файл Lab6.cpp

(Менеджер)

```
#include <windows.h>

#include <string>



#define ID_BTN_EXECUTE 101

#define ID_EDIT_N       102
```

```
#define ID_EDIT_MIN     103

#define ID_EDIT_MAX     104



#define WM_USER_START_OBJ3 (WM_USER + 11)



struct DataParams {

    int n;

    double minVal;

    double maxVal;

};



HWND hEditN, hEditMin, hEditMax;

HWND hObj2 = NULL, hObj3 = NULL;



void LaunchProcessIfNeeded(const wchar_t* exeName, const wchar_t* className,
HWND& hWnd) {
```

```
    hWnd = FindWindow(className, NULL);

    if (!hWnd) {

        STARTUPINFO si = { sizeof(si) };

        PROCESS_INFORMATION pi;

        if (CreateProcess(exeName, NULL, NULL, NULL, FALSE, 0, NULL, NULL, &si,
&pi)) {

            CloseHandle(pi.hThread);

            CloseHandle(pi.hProcess);

            int timeout = 0;

            while (!hWnd && timeout < 50) {

                Sleep(100);

                hWnd = FindWindow(className, NULL);

                timeout++;

            }

        }

    }
```

```
}



void AlignWindows(HWND hMain, HWND h2, HWND h3) {

    SetWindowPos(hMain, NULL, 50, 100, 320, 240, SWP_NOZORDER);

    if (h2) SetWindowPos(h2, NULL, 390, 100, 420, 400, SWP_NOZORDER |
SWP_SHOWWINDOW);

    if (h3) SetWindowPos(h3, NULL, 830, 100, 420, 400, SWP_NOZORDER |
SWP_SHOWWINDOW);

}



LRESULT CALLBACK WndProc(HWND hWnd, UINT msg, WPARAM wp, LPARAM lp) {

    switch (msg) {

    case WM_CREATE:

        CreateWindow(L"STATIC", L"Кількість n:", WS_CHILD | WS_VISIBLE, 20, 20,
100, 20, hWnd, NULL, NULL, NULL);

        hEditN = CreateWindow(L"EDIT", L"20", WS_CHILD | WS_VISIBLE | WS_BORDER
| ES_NUMBER, 130, 20, 140, 20, hWnd, (HMENU)ID_EDIT_N, NULL, NULL);
```

```
        CreateWindow(L"STATIC", L"Min:", WS_CHILD | WS_VISIBLE, 20, 50, 100, 20,
hWnd, NULL, NULL, NULL);

        hEditMin = CreateWindow(L"EDIT", L"-10.5", WS_CHILD | WS_VISIBLE |
WS_BORDER, 130, 50, 140, 20, hWnd, (HMENU)ID_EDIT_MIN, NULL, NULL);



        CreateWindow(L"STATIC", L"Max:", WS_CHILD | WS_VISIBLE, 20, 80, 100, 20,
hWnd, NULL, NULL, NULL);

        hEditMax = CreateWindow(L"EDIT", L"50.5", WS_CHILD | WS_VISIBLE |
WS_BORDER, 130, 80, 140, 20, hWnd, (HMENU)ID_EDIT_MAX, NULL, NULL);



        CreateWindow(L"BUTTON", L"Виконати", WS_CHILD | WS_VISIBLE |
BS_PUSHBUTTON, 20, 120, 250, 35, hWnd, (HMENU)ID_BTN_EXECUTE, NULL, NULL);

        break;



    case WM_COMMAND:

        if (LOWORD(wp) == ID_BTN_EXECUTE) {

            wchar_t bufN[32], bufMin[32], bufMax[32];

            GetWindowText(hEditN, bufN, 32);

            GetWindowText(hEditMin, bufMin, 32);
```

```
        GetWindowText(hEditMax, bufMax, 32);



        DataParams params;

        params.n = _wtoi(bufN);

        params.minVal = _wtof(bufMin);

        params.maxVal = _wtof(bufMax);



        LaunchProcessIfNeeded(L"Object2.exe", L"Object2Class", hObj2);

        LaunchProcessIfNeeded(L"Object3.exe", L"Object3Class", hObj3);



        AlignWindows(hWnd, hObj2, hObj3);



        if (hObj2) {

            COPYDATASTRUCT cds;

            cds.dwData = 1;

            cds.cbData = sizeof(params);
```

```
            cds.lpData = &params;

            SendMessage(hObj2, WM_COPYDATA, (WPARAM)hWnd, (LPARAM)&cds);

        }



        if (hObj3) {

            Sleep(50);

            PostMessage(hObj3, WM_USER_START_OBJ3, 0, 0);

        }

    }

    break;



case WM_DESTROY:

    if (hObj2) PostMessage(hObj2, WM_CLOSE, 0, 0);

    if (hObj3) PostMessage(hObj3, WM_CLOSE, 0, 0);

    PostQuitMessage(0);

    break;
```

```
    default:
        return DefWindowProc(hWnd, msg, wp, lp);
    }
    return 0;
}

int WINAPI WinMain(HINSTANCE hInst, HINSTANCE, LPSTR, int nCmdShow) {
    WNDCLASS wc = { 0 };
    wc.lpfnWndProc = WndProc;
    wc.hInstance = hInst;
    wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
    wc.lpszClassName = L"Lab6Class";
    RegisterClass(&wc);

    HWND hWnd = CreateWindow(L"Lab6Class", L"Lab6 - Менеджер (Варіант 2)",
```

```
        WS_OVERLAPPEDWINDOW & ~WS_THICKFRAME, 50, 100, 320, 240, NULL, NULL,
hInst, NULL);



    ShowWindow(hWnd, nCmdShow);

    UpdateWindow(hWnd);



    MSG msg;

    while (GetMessage(&msg, NULL, 0, 0)) {

        TranslateMessage(&msg);

        DispatchMessage(&msg);

    }

    return (int)msg.wParam;

}
```

Файл Object2.cpp

(Генератор)

```
#include <windows.h>

#include <vector>
```

```
#include <string>

#include <sstream>

#include <iomanip>

#include <random>

#include <cstdio>



#define WM_USER_OBJ2_DONE (WM_USER + 10)



struct DataParams {

    int n;

    double minVal;

    double maxVal;

};



std::vector<double> g_data;



void CopyToClipboard(const std::string& text) {

    if (OpenClipboard(NULL)) {

        EmptyClipboard();
```

```
        HGLOBAL hMem = GlobalAlloc(GMEM_MOVEABLE, text.size() + 1);
        if (hMem) {
            memcpy(GlobalLock(hMem), text.c_str(), text.size() + 1);
            GlobalUnlock(hMem);
            SetClipboardData(CF_TEXT, hMem);
        }
        CloseClipboard();
    }
}



LRESULT CALLBACK WndProc(HWND hWnd, UINT msg, WPARAM wp, LPARAM lp) {
    switch (msg) {
    case WM_COPYDATA: {
        PCOPYDATASTRUCT pcds = (PCOPYDATASTRUCT)lp;
        if (pcds && pcds->cbData == sizeof(DataParams)) {
            DataParams* p = (DataParams*)pcds->lpData;
            g_data.clear();



            std::random_device rd;
```

```
            std::mt19937 gen(rd());
            std::uniform_real_distribution<double> dis(p->minVal, p->maxVal);


            std::stringstream ss;
            for (int i = 0; i < p->n; ++i) {
                double val = dis(gen);
                g_data.push_back(val);
                ss << std::fixed << std::setprecision(2) << val << " ";
            }


            CopyToClipboard(ss.str());
            InvalidateRect(hWnd, NULL, TRUE);


            HWND hMain = (HWND)wp;
            if (hMain) PostMessage(hMain, WM_USER_OBJ2_DONE, 0, 0);
        }
        return TRUE;
    }
```

```
case WM_PAINT: {

    PAINTSTRUCT ps;

    HDC hdc = BeginPaint(hWnd, &ps);

    TextOut(hdc, 10, 10, L"Згенерований масив (Object2):", 29);



    int x = 10, y = 40;

    int colWidth = 70;

    int rowHeight = 20;



    for (size_t i = 0; i < g_data.size(); ++i) {

        wchar_t buf[32];

        swprintf(buf, L"%.2f", g_data[i]);

        TextOut(hdc, x, y, buf, wcslen(buf));



        y += rowHeight;

        if (y > 320) {

            y = 40;

            x += colWidth;

        }
```

```
        }

        EndPaint(hWnd, &ps);

        break;

    }



    case WM_DESTROY:

        PostQuitMessage(0);

        break;



    default:

        return DefWindowProc(hWnd, msg, wp, lp);

    }

    return 0;

}



int WINAPI WinMain(HINSTANCE hInst, HINSTANCE, LPSTR, int nCmdShow) {

    WNDCLASS wc = { 0 };

    wc.lpfnWndProc = WndProc;

    wc.hInstance = hInst;
```

```
    wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);

    wc.lpszClassName = L"Object2Class";

    RegisterClass(&wc);



    HWND hWnd = CreateWindow(L"Object2Class", L"Object2 - Генератор даних",

        WS_OVERLAPPEDWINDOW, 390, 100, 420, 400, NULL, NULL, hInst, NULL);



    ShowWindow(hWnd, nCmdShow);

    UpdateWindow(hWnd);



    MSG msg;

    while (GetMessage(&msg, NULL, 0, 0)) {

        TranslateMessage(&msg);

        DispatchMessage(&msg);

    }

    return (int)msg.wParam;

}
```

Файл Object3.cpp

(Сортувальник)

```
#include <windows.h>
```

```
#include <vector>

#include <string>

#include <sstream>

#include <algorithm>

#include <cstdio>



#define WM_USER_START_OBJ3 (WM_USER + 11)



std::vector<double> g_sortedData;



void ReadFromClipboard() {

    g_sortedData.clear();

    if (OpenClipboard(NULL)) {

        HANDLE hData = GetClipboardData(CF_TEXT);

        if (hData) {

            char* pszText = static_cast<char*>(GlobalLock(hData));

            if (pszText) {

                std::stringstream ss(pszText);

                double val;

                while (ss >> val) {

                    g_sortedData.push_back(val);

                }

                GlobalUnlock(hData);

            }

        }
```

```
        CloseClipboard();
    }
    std::sort(g_sortedData.begin(), g_sortedData.end());
}

LRESULT CALLBACK WndProc(HWND hWnd, UINT msg, WPARAM wp, LPARAM lp) {
    switch (msg) {
    case WM_USER_START_OBJ3:
        ReadFromClipboard();
        InvalidateRect(hWnd, NULL, TRUE);
        break;

    case WM_PAINT: {
        PAINTSTRUCT ps;
        HDC hdc = BeginPaint(hWnd, &ps);
        TextOut(hdc, 10, 10, L"Відсортований масив (Object3):", 30);

        int x = 10, y = 40;
        int colWidth = 70;
        int rowHeight = 20;

        for (size_t i = 0; i < g_sortedData.size(); ++i) {
            wchar_t buf[32];
            swprintf(buf, L"%.2f", g_sortedData[i]);
            TextOut(hdc, x, y, buf, wcslen(buf));
```

```
            y += rowHeight;
            if (y > 320) {
                y = 40;
                x += colWidth;
            }
        }
        EndPaint(hWnd, &ps);
        break;
    }

    case WM_DESTROY:
        PostQuitMessage(0);
        break;

    default:
        return DefWindowProc(hWnd, msg, wp, lp);
    }
    return 0;
}

int WINAPI WinMain(HINSTANCE hInst, HINSTANCE, LPSTR, int nCmdShow) {
    WNDCLASS wc = { 0 };
    wc.lpfnWndProc = WndProc;
    wc.hInstance = hInst;
```

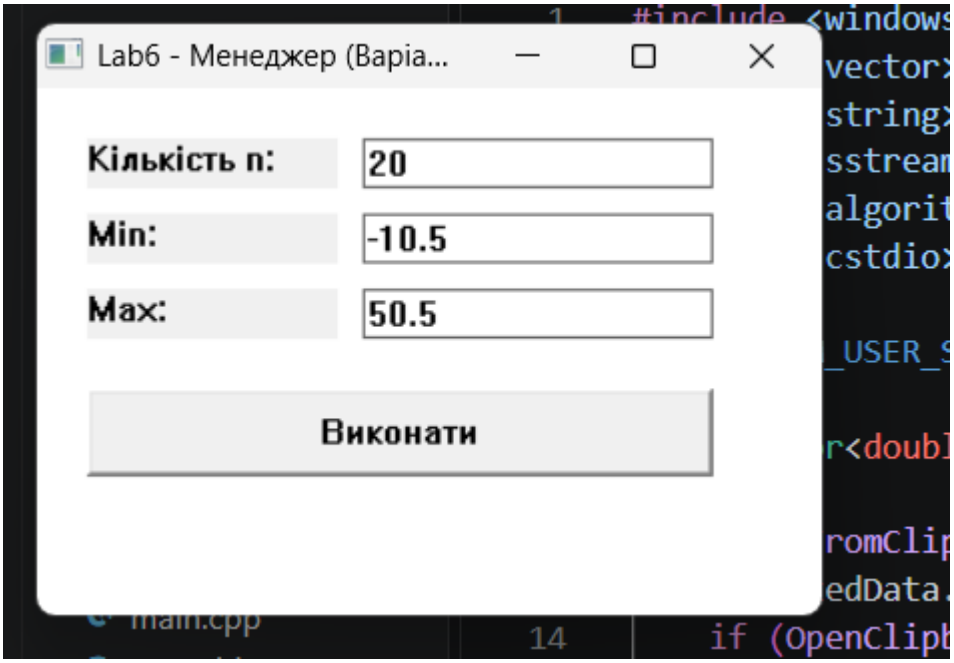
```
    wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
    wc.lpszClassName = L"Object3Class";
    RegisterClass(&wc);

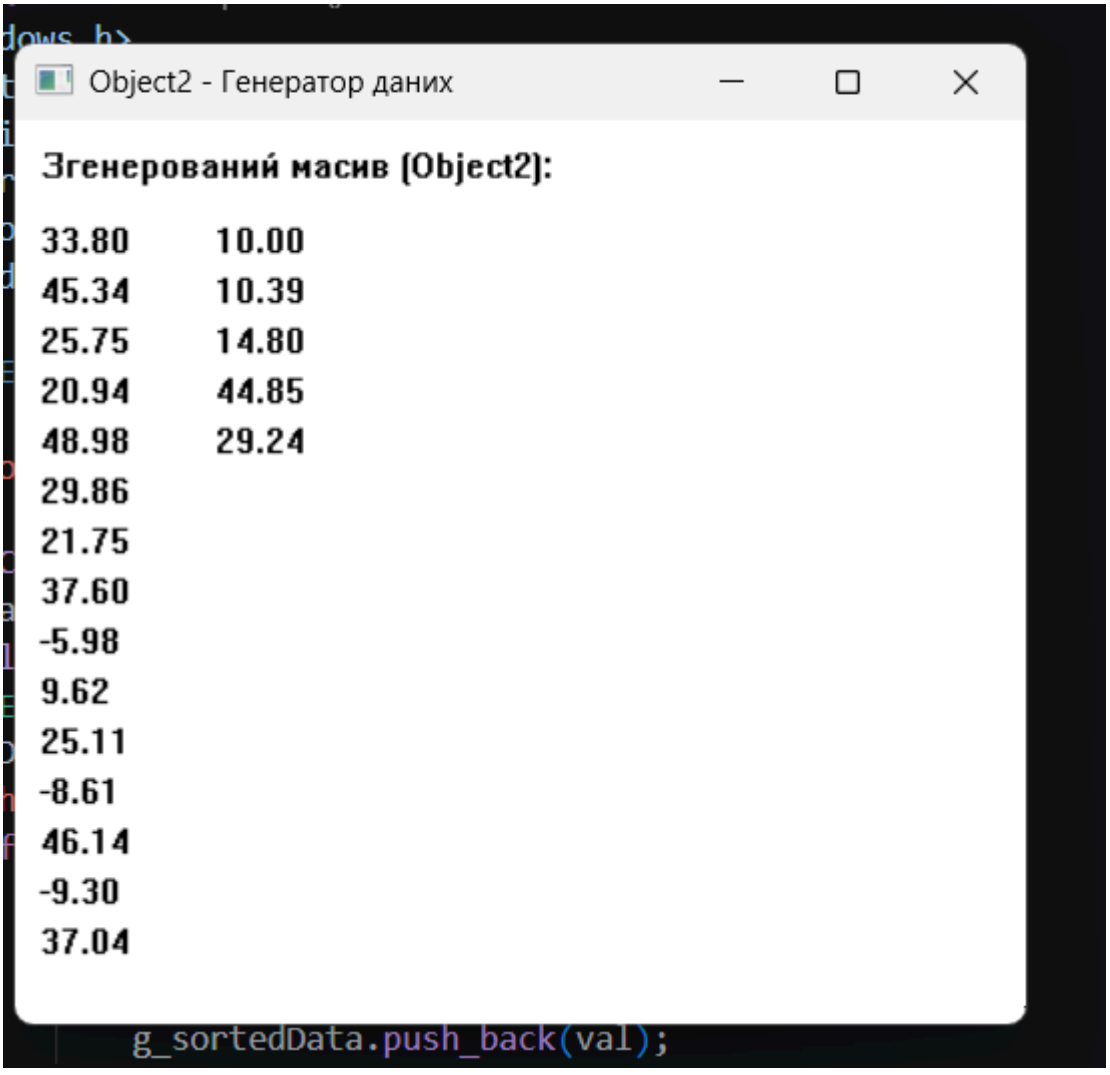
    HWND hWnd = CreateWindow(L"Object3Class", L"Object3 - Сортування",
        WS_OVERLAPPEDWINDOW, 830, 100, 420, 400, NULL, NULL, hInst, NULL);

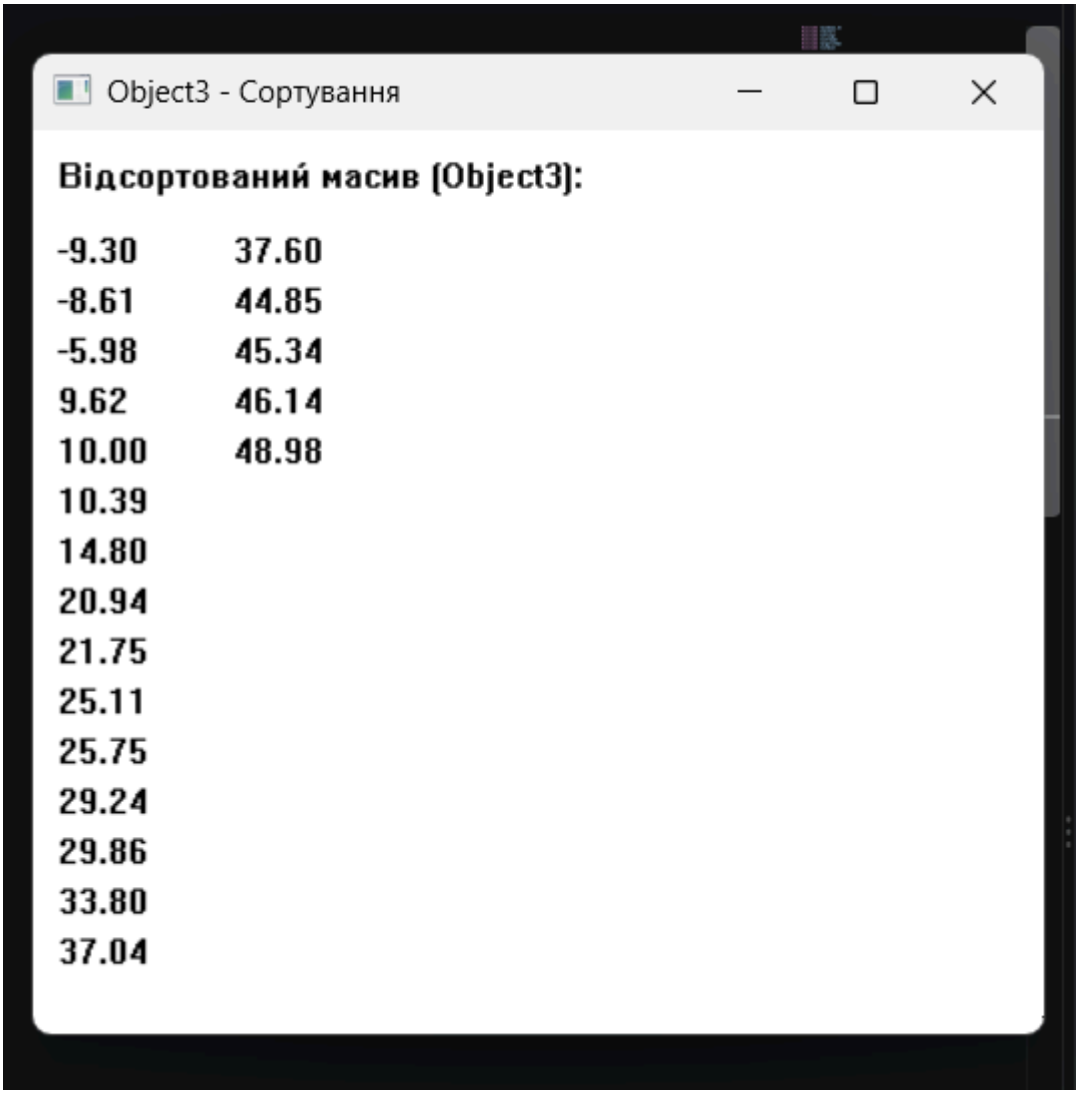
    ShowWindow(hWnd, nCmdShow);
    UpdateWindow(hWnd);

    MSG msg;
    while (GetMessage(&msg, NULL, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }
    return (int)msg.wParam;
}
```

Скріншоти:


Lab6 - Менеджер (Варіа...
Кількість n:
20
Min:
-10.5
Max:
50.5
Виконати


Object2 - Генератор даних
Згенерований масив (Object2):
33.80 10.00
45.34 10.39
25.75 14.80
20.94 44.85
48.98 29.24
29.86
21.75
37.60
-5.98
9.62
25.11
-8.61
46.14
-9.30
37.04



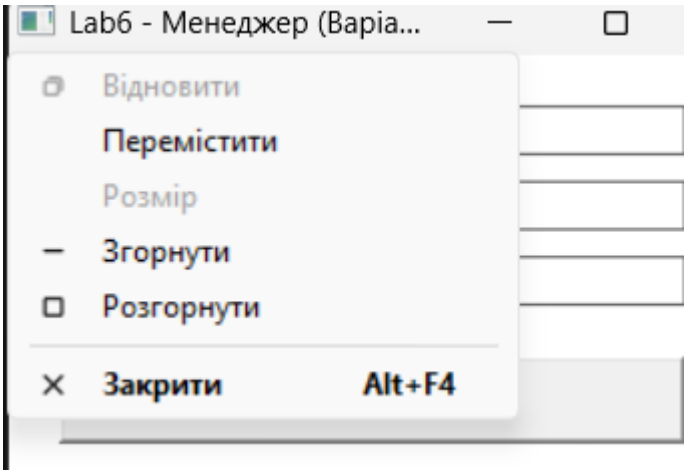
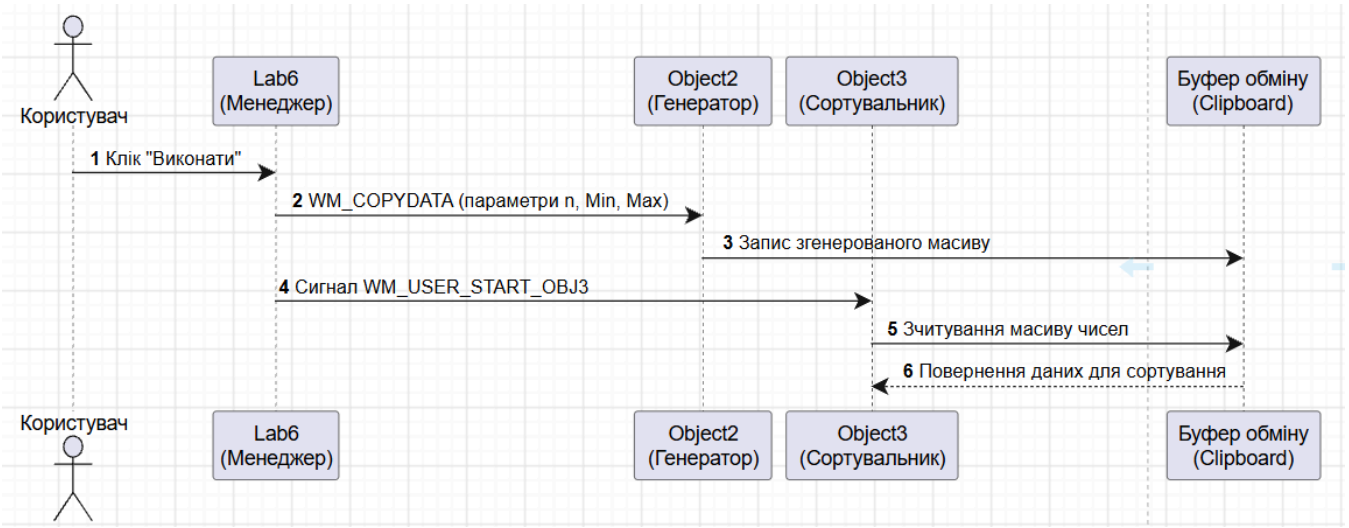


Схема послідовності надсилання-обробки повідомлень:



Висновки:

У ході виконання лабораторної роботи було практично опановано методи міжпроцесної взаємодії у системі Windows. Було реалізовано:

- Створення та керування сторонніми процессами через API CreateProcess та FindWindow.
- Пряму передачу структурованих даних між вікнами за допомогою системного повідомлення WM_COPYDATA.
- Застосування системного буфера обміну (Clipboard) як каналу обміну об'ємними даними між незалежними додатками.
- Потокове вирівнювання вікон декількох процесів для зручного відображення результатів користувачеві.